

FlowSpace

A Personal Google Workspace AI Dashboard

v1.2.0

Technical Report

March 2026

Author: Mohamed El Refaiy

Repository: github.com/moelrefaiy/flowspace

Stack: React 19 · Express · Tauri v2 · GLM-4.7

License: Open Source (BYO OAuth)

Table of Contents

1. Executive Summary
2. System Architecture
3. Server Layer (Express API)
4. AI Agent System
5. Frontend Application
6. Desktop Application (Tauri v2)
7. Authentication & Multi-Account
8. Data Flow & Caching
9. Tool System & Approval Workflow
10. Memory & Personalization
11. Build, Test & Deployment
12. Technology Stack
13. Security Model
14. Key Files & Metrics
15. Design Decisions & Trade-offs
16. Future Roadmap

1 Executive Summary

FlowSpace is a full-stack personal productivity application that consolidates Google Workspace services — Drive, Gmail, Calendar, and Tasks — into a single, AI-augmented dashboard. It delivers real-time workspace intelligence through an embedded conversational agent capable of reading from and writing to all connected Google services via a tool-calling loop.

The system ships in three deployment forms:

- **Web application** — Express + Vite on `localhost:3000`
- **macOS desktop application** — Tauri v2 with bundled Node.js runtime
- **Docker container** — Node 22 Alpine, single-command deployment

3,633 LINES — SERVER	23 AI TOOLS	34+ UI COMPONENTS
20+ API ENDPOINTS	~200 MB DESKTOP APP SIZE	v1.2.0 CURRENT RELEASE

Open-source model: FlowSpace is being prepared for open-source release with a "Bring Your Own OAuth" approach — users provide their own Google Cloud credentials. No centralized backend required.

2 System Architecture

FlowSpace follows a **two-layer monolith** pattern: a single Express.js server serves both the REST/SSE API and the frontend assets. In development, Vite's dev server is mounted as Express middleware for hot module replacement. In production, pre-built static files are served directly.



Component Breakdown

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend SPA	React 19, Tailwind CSS v4, Framer Motion	Dashboard UI, chat interface, email triage, calendar views

API Server	Express 4.21, TypeScript, tsx runtime	REST endpoints, SSE chat streaming, auth orchestration
AI Agent	OpenAI SDK → Z.AI (GLM-4.7)	Conversational AI with tool-calling across Google services
Desktop Shell	Tauri v2 (Rust), tauri-plugin-shell	macOS native window, server process lifecycle
CLI Bridge	gws (Google Workspace CLI)	OAuth flow, credential management, Google API execution

3 Server Layer (Express API)

The server (`server.ts` , 3,633 lines) is a single Express application that handles authentication, data aggregation, AI chat orchestration, and static file serving. It is the central coordinator for all application logic.

3.1 API Endpoints

CATEGORY	ENDPOINT	METHOD	DESCRIPTION
Authentication	<code>/api/auth/status</code>	GET	Auth state + user profile
	<code>/api/auth/gws-status</code>	GET	Check gws CLI installation & login
	<code>/api/auth/gws-install</code>	POST	Install gws CLI via npm
	<code>/api/auth/gws-login</code>	POST	Trigger Google OAuth flow
	<code>/api/auth/gws-import</code>	POST	Export & save credentials
	<code>/api/auth/logout</code>	POST	Clear auth state
	<code>/api/accounts/*</code>	GET/POST/DELETE	Multi-account management
Workspace Data	<code>/api/stats</code>	GET	Aggregated workspace statistics
	<code>/api/drive/recent</code>	GET	Recent Drive files
	<code>/api/gmail/threads</code>	GET	Gmail thread list with pagination
	<code>/api/calendar/upcoming</code>	GET	Calendar events (N-day window)
	<code>/api/tasks</code>	GET	Open tasks across all lists

AI & Chat	/api/chat	POST (SSE)	Streaming AI conversation with tools
	/api/chat/runs	GET	Run history (last 24h)
	/api/chat/runs/:id/approve	POST	Approve pending write action
Intelligence	/api/briefing	GET	AI-generated daily briefing (10-min cache)
	/api/followups	GET	Commitment tracking
	/api/activity/recent	GET	Merged activity feed

3.2 In-Memory Cache

All read endpoints use a 60-second TTL in-memory cache to minimize Google API calls. Cache keys are scoped per endpoint. The briefing endpoint uses a longer 10-minute TTL as its content changes less frequently and requires an LLM call to generate.

3.3 Run Tracking

Each chat interaction spawns a "run" tracked in an in-memory `Map<string, RunRecord>` with 24-hour auto-pruning. Runs transition through states: `queued` → `running` → `awaiting_approval` | `completed` | `failed`. The frontend's RunCenter component visualizes this pipeline.

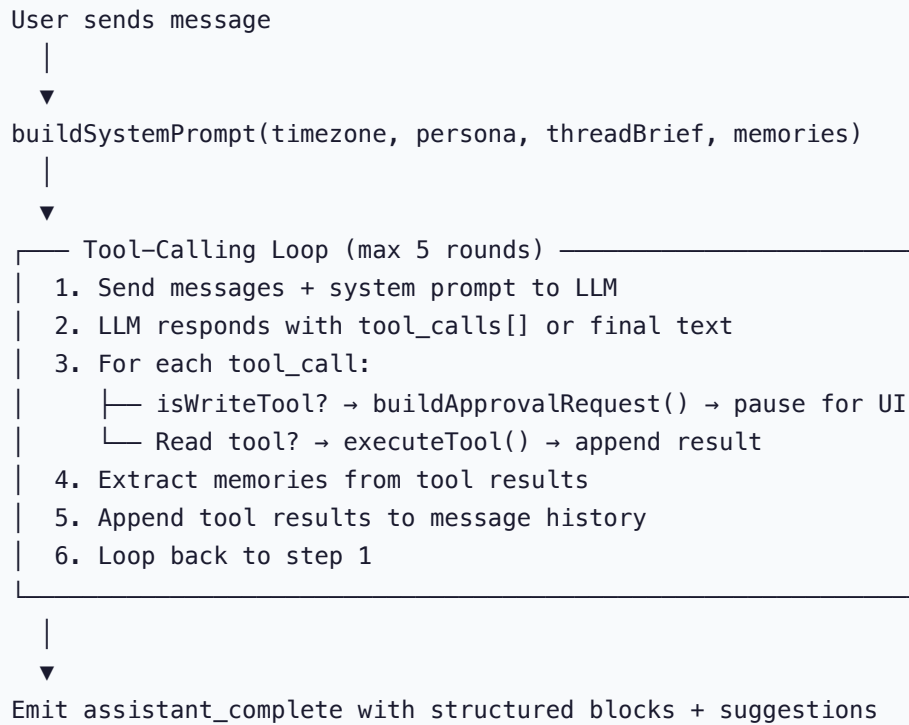
4 AI Agent System

The AI agent is the core differentiator of FlowSpace. It converts natural language instructions into concrete actions across Google Workspace, using a multi-round tool-calling loop.

4.1 LLM Integration

PROPERTY	VALUE
Provider	Z.AI (OpenAI-compatible endpoint)
Model	GLM-4.7 (by ZhipuAI / Alibaba)
API Base	<code>https://api.z.ai/api/coding/paas/v4</code>
SDK	OpenAI Node.js SDK v6.27
Max Tool Rounds	5 per conversation turn
Streaming	Server-Sent Events (SSE)
Configurable	Yes — LLM provider/model changeable via Settings

4.2 Chat Loop Architecture



4.3 System Prompt

The system prompt (~160 lines) is dynamically constructed per request. It includes: the current date/time in the user's timezone, persistent thread context (brief), retrieved long-term memories, tool usage guidelines, and output formatting rules (structured JSON blocks, 2-3 follow-up suggestions).

4.4 Tool Taxonomy

CATEGORY	TOOLS	APPROVAL REQUIRED
Read (14)	search_drive, search_emails, read_email, list_drive_files, list_calendar_events, list_tasks, docs_read, sheets_read, calendar_agenda, gmail_triage, review_overdue_tasks, standup_report, meeting_prep, weekly_digest	No
Write (9)	send_email, create_calendar_event, create_task, create_drive_folder, docs_write, sheets_create, sheets_append, sheets_update, drive_upload	Yes

Bulk Inbox (8)	archive_email_threads, trash_email_threads, restore_email_threads, mute_email_threads, mark_threads_read, apply_label_to_threads, unsubscribe_from_sender, create_gmail_filter	Yes
Meta (1)	create_tool (dynamic tool composition)	No

4.5 Tool Execution

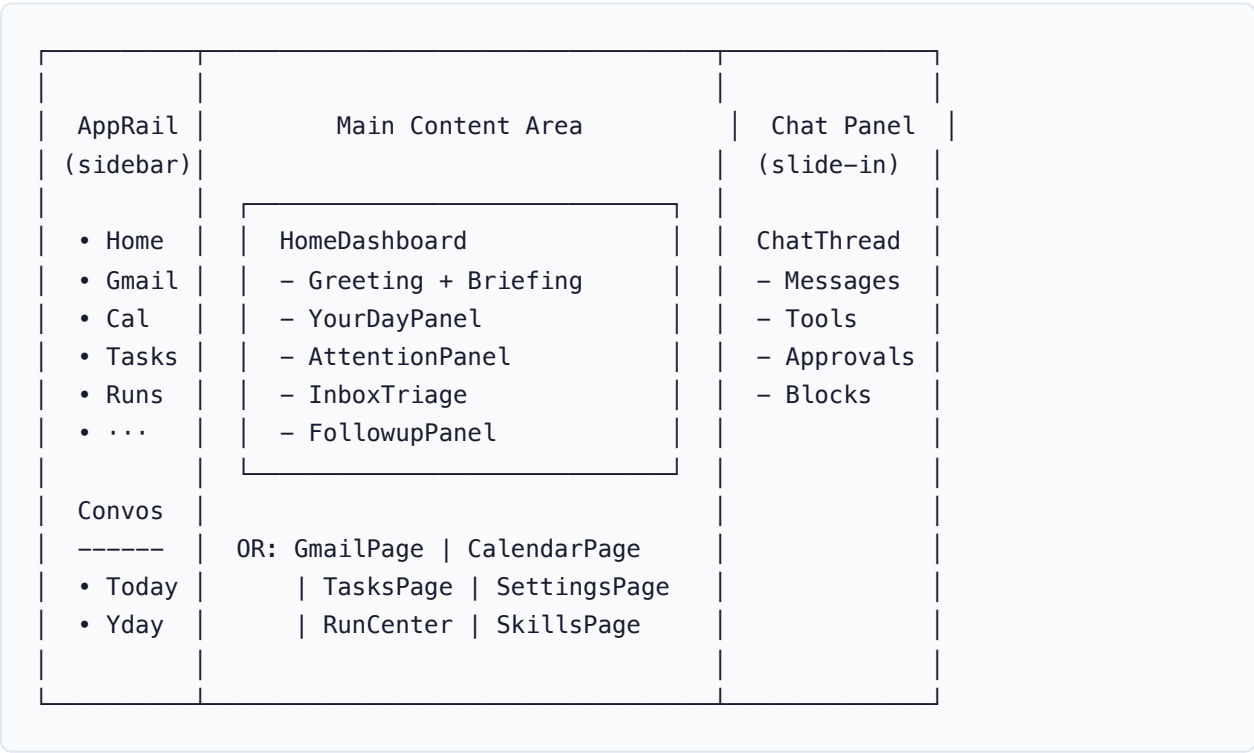
All Google Workspace operations are executed through the **gws CLI** as a subprocess (`execFile`, not shell). The access token is passed via the `GOOGLE_WORKSPACE_CLI_TOKEN` environment variable — never as a command-line argument. Each tool call has a 30-second timeout and supports abort signals.

5 Frontend Application

5.1 Technology

FRAMEWORK	React 19 with JSX transform
BUNDLER	Vite 6 with HMR
STYLING	Tailwind CSS v4 (utility-first) + CSS custom properties
ANIMATIONS	Framer Motion (motion) v12
ICONS	Lucide React v0.546
STATE	React Context (ChatContext) + localStorage persistence
ROUTING	In-app view switching (no React Router)

5.2 Application Layout



5.3 Key Components

COMPONENT	SIZE	RESPONSIBILITY
<code>ChatThread.tsx</code>	~45 KB	Message rendering with markdown, structured blocks (triage, agenda, sheets, drafts), tool activity log, approval UI
<code>SettingsPage.tsx</code>	~45 KB	LLM provider config, persona customization, account management
<code>ChatContext.tsx</code>	~31 KB	Conversations state, message history, approval workflow, thread groups
<code>RunCenter.tsx</code>	~23 KB	Run dashboard with status badges, error details, success metrics
<code>InboxTriage.tsx</code>	~18 KB	AI-categorized email buckets (urgent/review/low-priority), bulk actions
<code>HomeDashboard.tsx</code>	~17 KB	Daily briefing, attention items, calendar summary, activity feed
<code>AppRail.tsx</code>	~11 KB	Collapsible sidebar, conversation list, navigation, account switcher

5.4 Structured Response Blocks

The chat agent returns structured blocks that the frontend renders as rich UI cards:

BLOCK TYPE	RENDERING
<code>triage</code>	Email buckets with urgency levels and inline actions
<code>agenda</code>	Calendar events with attendees, locations, and linked docs
<code>email_draft</code>	Editable draft card with send/edit buttons
<code>sheet_data</code>	Spreadsheet table with headers and rows
<code>bulk_action_preview</code>	Preview of inbox actions with undo capability
<code>file_list</code> / <code>email_list</code> / <code>event_list</code>	Linked item lists with metadata
<code>fact_list</code>	Key-value pairs (stats, summaries)

6 Desktop Application (Tauri v2)

FlowSpace ships as a native macOS application using **Tauri v2**, a Rust-based framework that provides a lightweight alternative to Electron (~200 MB vs ~500 MB+).

6.1 Architecture

The Tauri shell (`lib.rs` , 112 lines) spawns the Express server as a child process, waits for it to become ready on port 3000, then navigates its WebView to `http://localhost:3000` . This approach allows the same codebase to run in both browser and desktop modes.

6.2 Bundled Resources

RESOURCE	PURPOSE
<code>resources/node</code>	Bundled Node.js binary (no system dependency)
<code>dist-server/server.mjs</code>	esbuild-bundled Express server
<code>dist/</code>	Vite-built frontend assets
<code>resources/client_secret.json</code>	Google OAuth client configuration
<code>resources/gws</code>	Bundled gws CLI binary

6.3 Server Lifecycle

App Launch

├─ Set FLOWSPACE_PRODUCTION=1

├─ Augment PATH (/opt/homebrew/bin, /usr/local/bin, ...)

├─ Spawn: node server.mjs

├─ Poll TCP:3000 (60 × 500ms = 30s timeout)

├─ Navigate WebView → http://localhost:3000

└─ On WindowEvent::Destroyed → kill server process

6.4 Build Targets

- `FlowSpace.app` — macOS application bundle
- `FlowSpace.dmg` — Disk image installer

- Code signing and notarization supported for distribution
- Minimum macOS version: 10.13 (High Sierra)

7 Authentication & Multi-Account

7.1 OAuth Flow

FlowSpace delegates OAuth entirely to the **gws CLI** (Google Workspace CLI), eliminating the need for users to create a Google Cloud project or configure OAuth client secrets.

```

User clicks "Sign in with Google"
  |
  ▼
[1] GET /api/auth/gws-status
    |— gws not installed? → POST /api/auth/gws-install
    |— gws not authenticated? → continue
    ▼
[2] POST /api/auth/gws-login
    |— Spawns: gws auth login -s drive,gmail,calendar,tasks,userinfo
    |— Opens browser → Google consent screen
    |— User authorizes → gws stores token
    ▼
[3] POST /api/auth/gws-import
    |— Spawns: gws auth export --unmasked
    |— Parses JSON credentials
    |— Saves to .gws-credentials.{accountKey}.json
    |— Upserts account record in .accounts.json
    ▼
[4] GET /api/auth/status → { authenticated: true, user: {...} }
  
```

7.2 Credential Hierarchy

When resolving credentials, the system tries sources in order:

1. **Active account** — `.accounts.json` → active account's credential file
2. **Legacy credentials** — `.gws-credentials.json` (single-account fallback)
3. **OAuth2 web flow** — `.tokens.json`
4. **Application Default Credentials** — `~/config/gcloud/application_default_credentials.json` (dev-only)

7.3 Multi-Account Support

FlowSpace supports multiple Google accounts. Each account is stored as an entry in `.accounts.json` with its own credential file. Users can switch accounts via the sidebar

account menu. All data (cache, conversations, preferences) is scoped per account.

8 Data Flow & Caching

8.1 Request Pipeline

```
Frontend Hook (useApiData)
  → fetch('/api/...')
  → Express Route Handler
    → getCached(key)?
      └─ HIT → return cached data (< 60s old)
      └─ MISS → getActiveAuthContext()
                → Google API call (googleapis SDK)
                → setCached(key, result)
                → return result
```

8.2 Cache Configuration

CACHE KEY	TTL	SOURCE
stats	60s	Parallel: Drive + Gmail + Calendar + Tasks
drive_recent	60s	Drive Files API (recent, sorted by viewedByMeTime)
gmail_recent	60s	Gmail Messages API (INBOX, maxResults=10)
calendar_upcoming	60s	Calendar Events API (next 7 days)
briefing	10 min	LLM-generated summary (expensive operation)
activity_recent	60s	Merged feed: Drive + Gmail + Calendar

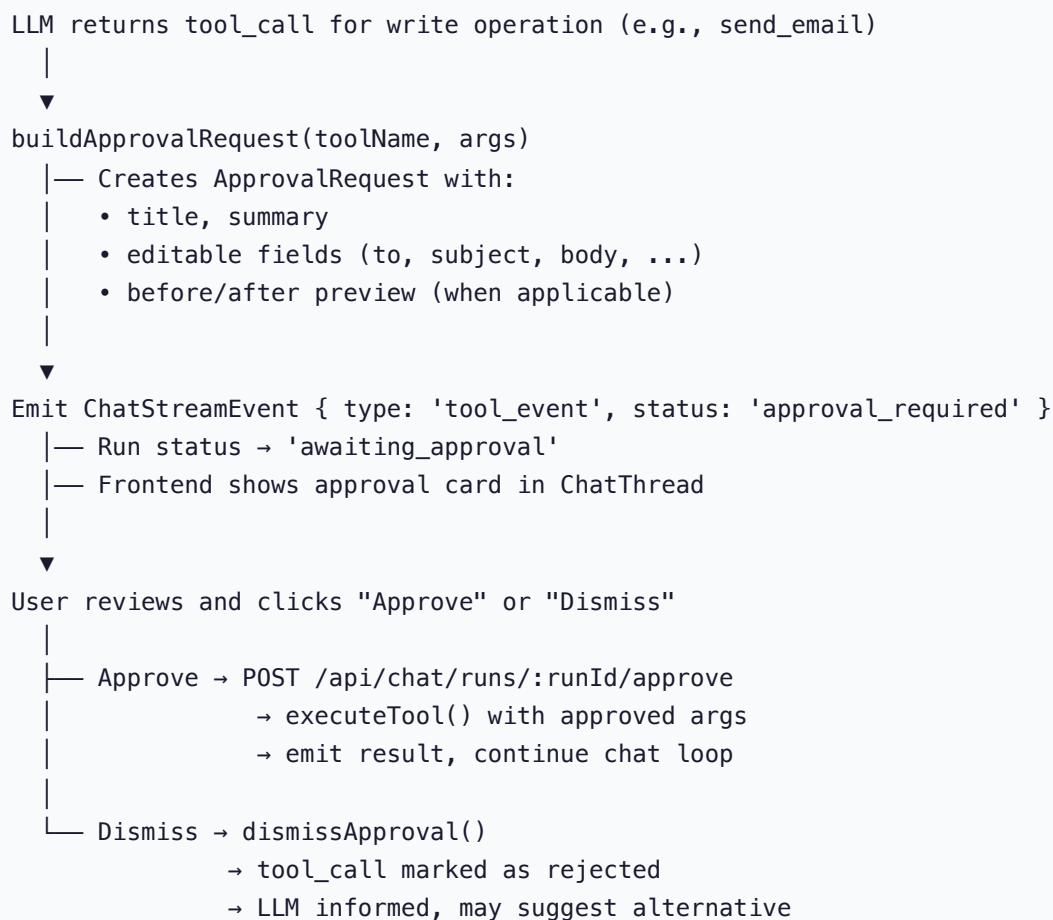
8.3 Frontend State

STORAGE	DATA	SCOPE
React Context	Active conversations, messages, pending approvals	Session
localStorage	Conversation history, preferences, collapsed state	Per-account, persistent
Server Memory	Run records, cache, auth state	Server lifetime
Filesystem	Credentials, memories, follow-ups, dynamic tools	Persistent (DATA_DIR)

9 Tool System & Approval Workflow

9.1 Approval Flow for Write Operations

All write tools (sending emails, creating events, modifying spreadsheets, etc.) require explicit user approval before execution. This prevents the AI from taking unintended actions on the user's behalf.



9.2 Dynamic Tool Composition

Users can create custom tools by composing existing tools into multi-step workflows. These are stored in `.dynamic-tools.json` and loaded into the tool registry at runtime.

```
// Example: Custom "expense tracker" tool
{
  "name": "log_expense",
  "description": "Log an expense to the team spreadsheet",
  "steps": [
    { "action": "sheets_read", "args": { "spreadsheet_id": "abc123" }, "outputKey": "data" },
    { "action": "sheets_append", "args": { "spreadsheet_id": "abc123", "values": "{data}" }, "outputKey": "data" }
  ]
}
```

10 Memory & Personalization

10.1 Long-Term Memory

The AI agent maintains a long-term memory system that persists across conversations. Memories are stored as JSON files in `.memory/{userHash}.json` within `DATA_DIR`.

COMPONENT	FILE	RESPONSIBILITY
Memory Store	<code>memory-store.ts</code>	CRUD operations on memory entries
Memory Retriever	<code>memory-retriever.ts</code>	Retrieve relevant memories for context injection
Memory Extractor	<code>memory-extractor.ts</code>	Extract memorable facts from tool results

10.2 Persona System

Users can customize the AI agent's personality and behavior through the Persona Settings. The persona prompt is injected into the system prompt, influencing tone, formality, and response style. Personas are stored per-account in `.persona.{accountKey}.json`.

10.3 Importance Feedback

The briefing system includes preference learning: users can mark items as more or less important, and these preferences influence future briefing generation. Feedback is stored via `/api/briefing/save-preference`.

11 Build, Test & Deployment

11.1 Build Pipeline

COMMAND	WHAT IT DOES
<code>make install</code>	<code>npm ci</code> — clean install from lockfile
<code>make dev</code>	<code>npm run dev</code> — Tailwind watch + tsx server (HMR)
<code>make build</code>	<code>npm run build</code> — Tailwind build + Vite production build
<code>make prod</code>	<code>NODE_ENV=production tsx server.ts</code>
<code>make typecheck</code>	<code>tsc --noEmit</code> — full type check
<code>make tauri-build</code>	Bundle sidecars + <code>tauri build</code> (signed + notarized)
<code>make docker</code>	Two-stage Docker build (Node 22 Alpine)

11.2 Testing

PROPERTY	VALUE
Framework	Vitest 4.0 + @vitest/coverage-v8
Test Environment	Node (default), jsdom (component tests)
Test Utilities	@testing-library/react, @testing-library/dom
Test Location	<code>src/**/*.__tests__/*</code> (co-located)
Coverage Target	80%+ on business logic
Commands	<code>npm test</code> , <code>npm run test:watch</code> , <code>npm run test:coverage</code>

11.3 Docker

```
# Stage 1: Build
FROM node:22-alpine AS builder
RUN apk add python3 make g++      # native deps (better-sqlite3, sharp)
COPY . .
RUN npm ci && npm run build

# Stage 2: Production
FROM node:22-alpine
COPY --from=builder /app/dist ./dist
COPY server.ts tsconfig.json src/agent/ ...
RUN npm ci --production
EXPOSE 3000
CMD ["npx", "tsx", "server.ts"]
```

11.4 Deployment Matrix

TARGET	METHOD	ARTIFACT
Local development	<code>make dev</code>	localhost:3000 (HMR)
Production web	<code>make prod</code> or Docker	Express on port 3000
macOS desktop	<code>make tauri-build</code>	FlowSpace.app + .dmg
Docker	<code>make docker</code> && <code>make docker-run</code>	Container image (~500 MB)

12 Technology Stack

CATEGORY	TECHNOLOGY	VERSION	PURPOSE
Core Runtime	Node.js	20+	Server runtime
	TypeScript	5.8.2	Type safety (full-stack)
	Express	4.21.2	HTTP server & API
	tsx	4.21.0	TypeScript execution (dev + prod)
	Vite	6.2.0	Frontend bundler + HMR dev server
Frontend	React	19.0.0	UI framework
	Tailwind CSS	4.1.14	Utility-first styling
	Framer Motion	12.23.24	Animations & transitions
	Lucide React	0.546.0	Icon library
Google APIs	googleapis	171.4.0	Drive, Gmail, Calendar, Tasks clients
	google-auth-library	10.6.1	OAuth2 token management
AI	OpenAI SDK	6.27.0	LLM client (OpenAI-compatible for Z.AI)
Desktop	Tauri	2.10.1	macOS app shell (Rust)
	tauri-plugin-shell	2.0	Subprocess management
Testing	Vitest	4.0.18	Test runner
	@vitest/coverage-v8	4.0.18	Code coverage
	@testing-library/react	16.3.2	Component testing
Optional	Supabase	2.99.1	Optional backend services
	better-sqlite3	12.4.1	Optional local database

13 Security Model

13.1 Authentication Security

- OAuth2 flow delegated to Google via gws CLI (no custom OAuth server)
- Credentials stored in user's home directory with filesystem permissions
- Access tokens refreshed automatically; refresh tokens never exposed to frontend
- Token passed to gws CLI via environment variable, never command-line argument

13.2 Data Privacy

- **Conversations:** Stored in browser localStorage (never sent to a server)
- **Memories:** Stored in user's local filesystem (DATA_DIR)
- **No telemetry:** No analytics, tracking, or data collection
- **No cloud backend:** All processing happens locally (except LLM API calls)

13.3 Write Protection

- All write operations require explicit user approval via the approval workflow
- Bulk inbox actions show preview of affected items before execution
- Undo capability for inbox actions via audit log
- Max 5 tool-calling rounds prevents runaway execution

13.4 Code-Level Safeguards

- No `eval()` or dynamic code execution
- Subprocess execution via `execFile` (not `exec /shell`) prevents injection
- Error messages sanitized before sending to client (no stack traces)
- CSP headers configured in Tauri for desktop builds
- `.gitignore` excludes all credential and token files

14 Key Files & Metrics

14.1 File Manifest

FILE	LINES	PURPOSE
server.ts	3,633	Express API, auth, chat orchestration, Google APIs
src/agent/tools.ts	2,252	23 tool definitions, approval builder, gws execution
src/App.tsx	1,323	Auth gate, layout, view routing
src/agent/chat.ts	806	Chat loop, LLM client, streaming, memory
src/services/api.ts	~700	Typed API client with all endpoint wrappers
src/components/ChatThread.tsx	~1,200	Message display, blocks, markdown
src/context/ChatContext.tsx	~850	State management, conversations
src-tauri/src/lib.rs	112	Tauri setup, server spawn
Makefile	107	Development and build targets
Dockerfile	32	Multi-stage container build

14.2 Project Metrics

~12K TOTAL LINES (TYPESCRIPT)	34+ REACT COMPONENTS	23 AI TOOL DEFINITIONS
20+ API ENDPOINTS	15 NPM DEPENDENCIES	3 DEPLOYMENT TARGETS

15 Design Decisions & Trade-offs

DECISION	RATIONALE	TRADE-OFF
Express monolith	Single process serves API + frontend. Simpler deployment, shared auth context, no CORS.	Cannot scale API and frontend independently.
gws CLI for auth	Zero GCP setup for users. No project creation, no client secrets to manage.	Runtime dependency on gws CLI binary.
In-memory cache	No database required. Fast dashboard loads. Simple implementation.	Data is eventually consistent (60s stale). Lost on server restart.
SSE for chat	Standard HTTP streaming. No WebSocket complexity. Works through proxies.	Unidirectional — approval responses require separate POST.
Tauri over Electron	~200 MB vs ~500 MB+. Native performance. Rust for reliability.	macOS-only currently. Smaller ecosystem than Electron.
localStorage for chat	Private by default. No cloud dependency. Immediate persistence.	Not synced across devices. Storage limited (~5-10 MB).
Tool approval flow	Prevents AI from taking unintended actions. User remains in control.	Adds friction to write operations. Cannot fully automate workflows.
GLM-4.7 default	Cost-effective. Good tool-calling support. OpenAI-compatible API.	Less capable than GPT-4/Claude for complex reasoning. Configurable as mitigation.

16 Future Roadmap

16.1 Open-Source Release

FlowSpace is being prepared for open-source distribution with a "Bring Your Own OAuth" model. Key changes include:

- **BYO OAuth credentials** — Users provide their own Google Cloud OAuth client
- `npx` **CLI installer** — One-command setup via `npx flowspace`

- **Fresh repository** — Clean git history for public release
- **Configurable LLM provider** — Already implemented (Settings page)

16.2 Planned Enhancements

- Cross-platform desktop builds (Windows, Linux via Tauri v2)
- Plugin system for third-party tool extensions
- Collaborative features (shared workspace views)
- Offline mode with SQLite-backed cache
- Enhanced memory system with semantic search

FlowSpace Technical Report — v1.2.0 — March 2026

Generated from source analysis of the FlowSpace repository.

For questions or contributions, see the project repository.